

Mini-TP 6 — Architecture MVVM

Feuille de l'etudiant

Nom : RASETRATINJANAHAARY Zotina

ETU : ETU002597

Etape 1 - Lire et predire (AVANT de lancer)

J'ai lu ProduitsViewModel.kt et MainActivity.kt. Le ViewModel est la couche manquante — il porte l'état avec StateFlow. Les écrans observent l'état via collectAsState(). J'ai prédit sans lancer.

Prediction	Votre reponse
P1 — AVANT les TODO : que fait le bouton « Ajouter 1 kg au panier » de l'écran de détail, et pourquoi ?	Rien. Le bouton appelle <code>viewModel.adderAuPanier(1)</code> mais la fonction est vide (elle ne fait rien). L'état ne change pas, le panier reste à 0.
P2 — APRÈS les TODO : vous ajoutez 3 kg au panier, puis vous TOUREZ L'ÉCRAN. Qu'affiche le panier, et pourquoi ?	Le panier affiche 3 kg. Le ViewModel survit à la rotation — il est au-dessus de l'Activity et n'est pas détruit quand l'Activity est recréée. L'état est toujours là.

Etape 2 - Les deux TODO du ViewModel

TODO 1 — l'état :

J'ai remplacé la ligne provisoire par le duo `_uiState` / `uiState` :

```
private val _uiState = MutableStateFlow(EtatUi(produits = produits))
val uiState: StateFlow<EtatUi> = _uiState
```

`_uiState` est privé et mutable (seul le ViewModel écrit). `uiState` est public et en lecture seule (l'UI ne fait qu'observer).

TODO 2 — l'événement :

J'ai rempli `ajouterAuPanier()` :

```
fun ajouterAuPanier(poidsKg: Int) {
    _uiState.update { etat ->
        etat.copy(poidsPanierKg = etat.poidsPanierKg + poidsKg)
    }
}
```

Le StateFlow émet un nouvel état, l'UI se recompose automatiquement.

Verification :

- [x] Le panier se met à jour quand on clique sur « Ajouter 1 kg »
- [x] Le panier SURVIT à la rotation (l'affichage garde la valeur)

Question de contrôle : pourquoi `uiState` est-il déclaré `StateFlow` et non `MutableStateFlow` ?

`StateFlow` est en lecture seule. L'interface ne peut que lire l'état, pas l'écrire. Ça empêche l'UI de modifier

directement le `stateFlow` — seul le `ViewModel` a le droit d'écrire dans `_uiState`. Le flux unidirectionnel est garanti par les types.

Etape 3 - Casser le flux pour comprendre

J'ai décommenté `var poidsTriche = 0` en haut du fichier et le bloc « triche » dans `EcranDetail`.

7. Cliquer plusieurs fois sur « Ajouter 1 kg (hors circuit) » :

L'affichage ne change pas. Le texte reste à `Triche : 0 kg`. `poidsTriche` est incrémenté en mémoire mais `Compose` ne le voit pas — ce n'est ni un `StateFlow` ni un `remember`. Pas de recomposition déclenchée.

8. Maintenant, TOURNER L'ÉCRAN :

La valeur affichée saute d'un coup et montre tous les clics cumulés. La rotation force une recomposition complète, et `poidsTriche` a bien été incrémenté en mémoire pendant les clics. Donc la nouvelle instance l'affiche avec la valeur accumulée.

9. Explication en trois lignes :

- `poidsTriche` est un simple `var`, pas un `StateFlow` observé par `Compose` — les clics ne déclenchent pas de recomposition, l'écran ne se met pas à jour.
- Mais la valeur réelle est bien incrémentée en mémoire, donc quand on tourne l'écran (recomposition forcée), la valeur accumulée apparaît d'un coup.
- C'est différent du panier normal (`StateFlow`) qui se met à jour à chaque clic immédiatement.

Etape 4 - Voie ouverte : trier une critique IA

J'ai soumis `ProduitsViewModel.kt` complet à l'IA pour une critique. J'ai trié chaque remarque.

Synthèse du tri en 3 lignes :

- Remarque la plus pertinente : l'IA a signalé qu'il faudrait initialiser `produits` depuis une source externe plutôt que de la passer directement dans le constructeur — c'est pertinent car le vrai repository viendra en séance 7.
- Remarque moins pertinente : l'IA a suggéré d'ajouter des tests unitaires et une injection de dépendances — hors périmètre du jour, on abordera ça plus tard.
- Le `ViewModel` fonctionne correctement avec les deux `TODO` : le flux unidirectionnel est bien respecté grâce au duo `_uiState / uiState`.

Commentaires supplémentaires

Le `ViewModel` répond à la question ouverte de la séance 3 : l'état survit à la rotation parce que le `ViewModel` vit plus longtemps que l'`Activity`. C'est la différence fondamentale avec `remember` de la séance 4.

Le flux unidirectionnel fonctionne : l'état descend du `ViewModel` vers l'UI, les événements remontent de l'UI vers le `ViewModel`. Le bloc triche montre ce qui se passe quand on casse ce flux — l'interface ne se met pas à jour car `Compose` ne peut pas observer la variable.